PART 6

Tool Use

Five chapters on the mechanism that turns a text model into a system that acts: the loop and who runs the code, the JSON Schema that describes an operation, the words that decide whether it is called, the contract a failing tool owes its caller, and the orchestration of many tools across many turns.

IN THIS PART

- 31. How Tool Use Works
- 32. Designing Tool Schemas with JSON Schema
- 33. Tool Descriptions, Naming and Affordances
- 34. Error Contracts, Validation and Idempotency
- 35. Tool Chaining and Orchestration

CHAPTER 31

How Tool Use Works

In this chapter

Introduction: The Tenth Tool . . . 4
31.1 Core Concepts: The Contract . . . 5
31.2 Where Tools Run: Three Buckets . . . 6
31.3 The Client Loop . . . 7
31.4 The Server-Side Loop . . . 8
31.5 Formatting a Continuation . . . 8
31.6 Parallel Tool Use . . . 9
31.7 When Tool Use Is the Wrong Answer . . . 10
31.8 The Economics of a Tool-Using Turn . . . 10
31.9 Architecture: Where the Loop Lives . . . 11
31.10 Figures . . . 11
31.11 Examples . . . 12
31.18 Code Walkthrough: A Loop That Refuses, and a Linter for What Is Legal . . . 13
31.19 Mini Project: The Loop, the Gate, the Linter and the Fan-out Cliff . . . 14
31.20 Hands-on Labs . . . 17
31.21 Lab Solutions . . . 17
31.22 Best Practices . . . 17
31.23 Common Mistakes . . . 18
31.24 Performance Considerations . . . 19
31.25 Security Considerations . . . 20
31.26 Real Production Examples . . . 20
31.27 Certification Notes . . . 21
31.28 Twenty Practice Questions . . . 21
31.29 Answers . . . 22
31.30 Ten Practical Exercises . . . 23
31.31 Five Challenge Labs . . . 23
31.32 Ten Interview Questions . . . 24
31.33 Summary . . . 24
31.34 Glossary . . . 25

31.35 Further Reading 26

Part 6 of 13 · Chapter 31 of 64. Next: Chapter 32 — Designing Tool Schemas with JSON Schema.

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. State the tool-use contract precisely — what the model emits, what your code does, and why **the model never executes anything**.
2. Classify any tool as user-defined client, Anthropic-schema client or server-executed, and say what each bucket makes you responsible for.
3. Write the client agentic loop as a state machine over `stop_reason`, handling all seven values rather than the one everybody handles.
4. Distinguish the client loop from the **server-side loop**, and continue a `pause_turn` correctly.
5. Format a continuation that the API will accept, and reproduce every documented 400 offline.
6. Decide whether to run parallel calls concurrently or sequentially, and return results in the one shape that keeps parallelism alive.
7. Use `disable_parallel_tool_use` in the right place — inside `tool_choice`, not at the top level — and predict its effect for each `tool_choice` type.
8. Price a tool-using turn in round trips, tokens and cache hits, and identify the fan-out at which parallelism starts costing money.

NOTE

Prerequisites

Chapter 10 for the capability triage that decides whether a task needs a tool at all. **Chapter 16** for the Messages API, typed content blocks and statelessness as a quadratic cost. **Chapter 17** for fine-grained tool streaming and the `INVALID_JSON` contract. **Chapter 18** for the rule that **a tool-use loop is one assistant turn**. **Chapter 23** for `strict: true` and constrained decoding on a tool's `input`. **Chapter 28** for all seven stop reasons, the `tool_use` continuation contract and the unresolved `server_tool_use` block. **Chapter 29** for what to retry. **Chapter 30** for the 20-block cache lookback window and caching tool definitions.

Deferred by name: **JSON Schema and the supported subset** is **Chapter 32**; **naming, descriptions and affordances** is **Chapter 33**; **error contracts, validation and idempotency** is **Chapter 34**; **chaining and orchestration across many turns** is **Chapter 35**. **MCP**, which is how tools arrive from somewhere other than your own codebase, is **Part 7 (Chapters 36–40)**. **Indirect prompt injection through tool results** is treated properly in **Chapter 58**.

Introduction: The Tenth Tool

An agent that reads a codebase fires tools in parallel. It asks for nine files at once, gets nine results, reasons, asks for nine more. The team is pleased with it: fewer round trips than the sequential version, and the latency shows it.

Someone widens the fan-out to ten. Round trips do not change — the task still takes four requests. Latency does not change. The input bill **doubles**: 48,004 base-input units become 100,140, a **108.6 per cent** increase, measured in $31.19.

Nothing about tool use changed. What changed was the prompt cache. Chapter 30 established that the lookback window is **20 blocks**, and that a turn may add at most 19 before a single end-of-prompt breakpoint stops finding the previous write. A parallel fan-out of `k` tools adds `k` `tool_use` blocks in the assistant turn and `k` `tool_result` blocks in the user turn — **2k blocks per round trip**. At `k = 9` that is 18 blocks and every turn hits. At `k = 10` it is 20, and the cache never hits again for the rest of the conversation.

The largest safe fan-out is **nine**, and it is not written down anywhere, because it is the interaction of two features documented on different pages. That is the character of this chapter and of Part 6 generally: the tool-use mechanism is small and well specified, and almost everything expensive about it lives in how it meets the rest of the API.

There is a second failure in the same family, and it is worse because nothing rejects it. The documentation states that returning each tool result in its own user message *"'teaches' Claude to avoid parallel calls."* Every request is valid. Every response is a 200. The model simply stops batching, and a task that took four round trips starts taking twenty-five — **6.25 times the input tokens and 6.59 times the wall clock**, both measured in $31.19. There is no error to catch, no metric that moves, and no line in the logs. The only thing that finds it is a linter you wrote yourself, which $31.18 does.

So this chapter builds three things: a loop that handles all seven stop reasons rather than the one everybody handles, a gate that reproduces every documented continuation 400 without a key, and a linter for the failure that is legal.

31.1 Core Concepts: The Contract

DEFINITION

Tool use

A contract in which **you** declare what operations exist and what shape their inputs take, and **Claude** decides when to call them. *"The model never executes anything on its own. It emits a structured request, your code (or Anthropic's servers) runs the operation, and the result flows back into the conversation."*

DEFINITION

`tool_use` block

An assistant content block carrying `id` (unique, used to match the result), `name` (the tool) and `input` (a JSON object conforming to the tool's `input_schema`).

DEFINITION

`tool_result` block

A **user** content block carrying `tool_use_id`, an optional `content` (a string, or a list of `text` / `image` / `document` / `search_result` blocks) and an optional `is_error: true`.

DEFINITION

Client tool

A tool your application executes. The response contains a `tool_use` block; you run the code and return a `tool_result`.

DEFINITION

Server tool

A tool Anthropic executes — `web_search`, `web_fetch`, `code_execution`, `tool_search`. *"You never construct a* `tool_result` *block for these tools."* The response carries `server_tool_use` blocks showing what ran.

DEFINITION

Agentic loop

The `while stop_reason == "tool_use"` cycle: request, `tool_use`, execute, `tool_result`, repeat. Chapter 18's rule stands — the whole cycle is **one assistant turn**.

DEFINITION

Parallel tool use

Several `tool_use` blocks in one assistant turn. On by default for Claude 4 and later. *"The API doesn't prescribe an execution order"* — how you run them is your decision.

The sentence to hold on to is the one about who runs code. Chapter 25 gave the running thread: **the model proposes, your system disposes**. Tool use is that thread made into a protocol. Claude cannot reach your database, cannot send your email and cannot delete your file. It can only ask, in a typed shape you defined, and every consequence in the world happens because your code chose to act on the request. Chapter 34 turns that observation into an authorisation design; here it is simply the reason the mechanism is safe enough to be worth having.

31.2 Where Tools Run: Three Buckets

"The primary axis along which tools differ is where the code executes." Three buckets, and the bucket decides what you are responsible for.

User-defined tools (client-executed). You write the schema, run the code, return the result. *"The vast majority of tool-use traffic"* is this. Claude never sees your implementation — *"it only sees the schema you provided and the result you returned"*, which is why Chapter 33 spends a whole chapter on the words in the schema.

Anthropic-schema tools (client-executed). `memory`, `bash`, `text_editor` and `computer`. Anthropic publishes the schema; **you still execute**. The execution model is identical to a user-defined tool. The reason to use one is not convenience: *"these schemas are trained-in. Claude has been optimized on thousands of successful trajectories that use these exact tool signatures, so it calls them more reliably and*

recovers from errors more gracefully than it would with a custom tool that does the same thing." Chapter 7's three training stages explain why that is true and Chapter 33 explains what to do about it — **the schema is the interface the model already expects**, so a custom `run_shell_command` tool is measurably worse than `bash` at the same job.

Server-executed tools. `web_search`, `web_fetch`, `code_execution`, `tool_search`. You enable them; Anthropic runs them. *"In the common case, execution is already complete by the time you see them, and your application's job is to enable the tool and read the final answer rather than to participate in the execution loop."*

Bucket	Who writes the schema	Who executes	Do you build `tool_result`?
User-defined	You	You	Yes
Anthropic-schema	Anthropic	You	Yes
Server-executed	Anthropic	Anthropic	**No**

Two exceptions to "server tools just work", both from Chapter 28. A turn can pause (`pause_turn`), and a turn can hand you a `server_tool_use` block with **no matching result**, which happens when Claude calls a server tool and a client tool in the same group of parallel calls. In that second case the response's `stop_reason` is `tool_use` — there is no distinct signal — and, as Chapter 28 recorded, *"there is no other marker for the state; detect it by checking each block's* `id` *for a matching result block."*

31.3 The Client Loop

The canonical shape, quoted:

1. Send a request with your `tools` array and the user message.
2. Claude responds with `stop_reason: "tool_use"` and one or more `tool_use` blocks.
3. Execute each tool. Format the outputs as `tool_result` blocks.
4. Send a new request containing the original messages, the assistant's response, and a user message with the `tool_result` blocks.
5. Repeat from step 2 while `stop_reason` is `"tool_use"`.

"The loop exits on any other stop reason (`"end_turn"`*,* `"max_tokens"`*,* `"stop_sequence"`*, or* `"refusal"`*), which means Claude has either produced a final answer or stopped for another reason that your application should handle."*

"Should handle" is doing a great deal of work in that sentence, and Chapter 28 supplies the detail the loop needs. Seven stop reasons, three distinct behaviours:

`stop_reason`	Loop action	Why
`tool_use`	**RUN_TOOLS**	The only value that continues *your* loop
`pause_turn`	**RESEND**	The **server's** loop hit its cap; nothing of yours to run
`end_turn`	DONE	Complete

stop_reason	Loop action	Why
stop_sequence	DONE	Complete
max_tokens	**HALT**	Chapter 27: may contain an **incomplete tool use block — do not execute it**
refusal	**HALT**	Chapter 25: needs a *different model*, not a retry
model_context_window_exceeded	**HALT**	Chapter 28: typed only in the SDKs' beta namespace

A loop written as `while response.stop_reason == "tool_use"` is correct for the happy path and silently wrong for three of the other six. On `max_tokens` it falls out of the loop with a truncated answer and no indication that a tool call was cut in half. On `refusal` it falls out with `content` that may be **empty** — Chapter 25's measurement — and reads as an ordinary short answer. On `pause_turn` it exits treating an unfinished turn as finished. §31.18's loop is a table lookup that raises on anything unlisted, for the reason Chapter 26 gave: the versioning contract permits **new enum values**, so a loop that falls through on an unknown stop reason has a bug with a delivery date.

31.4 The Server-Side Loop

Server tools run their own loop inside Anthropic's infrastructure: *"A single request from your application might trigger several web searches or code executions before a response comes back."*

That loop has an iteration cap — Chapter 28 verified the documented default as **10 iterations per request** — and when it is reached the response carries `stop_reason: "pause_turn"`. *"A paused turn means the work isn't finished; re-send the conversation (including the paused response) to let the model continue where it left off."*

Two properties are worth stating plainly because they are easy to invert. The client loop is driven by **your** inability to run code inside Anthropic's process; the server loop is driven by Anthropic's need to bound work inside one HTTP request. They are not the same loop at different scales, and `pause_turn` never means one of your tools is waiting. Chapter 28 quoted the documentation exactly: *"A response that leaves a client `tool_use` block waiting on you never has a `stop_reason` of `pause_turn`."*

Chapter 30 added one modification for the batch path: because there is no open connection to maintain, *"the batch loop runs more iterations per turn than a synchronous request before it returns `stop_reason: "pause_turn"`."* How many is not published, which this book records as **UNKNOWN** rather than assuming it is still ten.

31.5 Formatting a Continuation

The continuation is where most of the 400s live. Four rules, all documented, all reproduced offline in §31.19.

One — results immediately follow their calls. *"Tool result blocks must immediately follow their corresponding tool use blocks in the message history. You cannot include any messages between the*

assistant's tool use message and the user's tool result message."

Two — results come first in the content array. *"In the user message containing tool results, the* `tool_result` *blocks must come FIRST in the content array. Any text must come AFTER all tool results."* Text before a result produces `tool_use ids were found without tool_result blocks immediately after`.

Three — one result per call, and only for calls that happened. Every `tool_use` id needs exactly one `tool_result`. If you chose not to run a call, *"still return a* `tool_result` *for it with* `is_error: true` *and a brief explanation."* That is a genuinely useful affordance: a sequential executor that stops on first failure can still produce a legal continuation.

Four — if a server tool is unresolved, the user message must contain nothing but results. *"Text after the results ends the turn early; for a server tool Claude called directly, the request then fails with a 400 error that names the unresolved server tool."* This is the rule that makes trailing text conditionally illegal: legal after a purely client-tool turn, a 400 after a mixed one. Your formatter cannot know which without inspecting the response.

And one rule about the request rather than the message list: **the** `tools` **array must be unchanged**. Dropping a tool between turns produces `but no 'web_search' tool was provided`. Chapter 30 supplies the second reason to keep it stable — any edit to `tools` invalidates the entire cache, including `system` and `messages`.

> WARNING
>
> **Tool results are untrusted input**
>
> *"Tool results often carry content from sources outside your control: web pages, inbound email, user uploads, third-party APIs. Treat that content as untrusted: an attacker who can influence it may embed instructions that try to redirect Claude (indirect prompt injection). Keep untrusted content inside* `tool_result` *blocks rather than* `system` *prompts or plain user* `text` *blocks."* Chapter 21 established the system prompt as an authority boundary; this is the same argument from the other end. Chapter 22 measured how readily ordinary payloads speak your markup. Chapter 58 hardens it.

31.6 Parallel Tool Use

"By default, Claude may call multiple tools in a single response." Claude 4 and later do it *"when a request benefits from multiple tools"*, and it can be encouraged by prompting — Chapter 20's rules apply unchanged.

Execution order is yours. *"The API doesn't prescribe an execution order: you can run the calls concurrently (*`Promise.all`*,* `asyncio.gather`*), sequentially in the order they appear, or in any combination that suits your tools."* The documented guidance is exactly what you would tell a colleague: *"Independent, read-only operations are usually safe to run in parallel… Tools with side effects, shared state, or ordering requirements might be better run sequentially."*

Return shape is not yours. All results go in **one** user message, results before text, one per `tool_use_id`. This is where the interesting failure lives, because the wrong shape is legal:

```
[
  {"role": "assistant", "content": ["tool_use_1", "tool_use_2"]},
```

```
  {"role": "user", "content": ["tool_result_1"]},
  {"role": "user", "content": ["tool_result_2"]}
]
```

Nothing rejects that. The documentation's verdict is that it *"'teaches' Claude to avoid parallel calls"* — the history becomes an in-context example of one-call-per-turn, and Chapter 5's induction heads explain the mechanism: the model continues the pattern it can see. §31.19 measures what the degradation costs on a 24-tool task: fan-out collapsing from 8 to 1 is **6.25× the input tokens** and **6.59× the wall clock**.

Disabling it is a `tool_choice` field, not a top-level parameter. *"To turn it off, set `disable_parallel_tool_use: true` inside the `tool_choice` object. It is not a top-level request parameter."* The effect depends on the type:

`tool_choice` type	With `disable_parallel_tool_use: true`
`auto` (default)	**At most one** tool call; Claude may still answer in plain text
`any`	**Exactly one** tool call
`tool`	**Exactly one** call, of the named tool

Chapter 30's invalidation table adds the cost: changing `tool_choice` or `disable_parallel_tool_use` invalidates the **messages** cache. Toggling it per request is a way of never caching anything after the tools.

31.7 When Tool Use Is the Wrong Answer

Chapter 10 built a triage — information, exact execution, external state, checkability, with prompting as the fall-through — and this section is the tool-shaped half of it, stated by the documentation itself.

Tool use fits **actions with side effects** (*"the model can describe these actions, but only a tool can perform them"*), **fresh or external data**, **structured, guaranteed-shape outputs**, and **calling into existing systems**.

The best diagnostic in the whole tool-use documentation is this one: *"A clear sign that you should be using tools: if you're writing a regex to extract a decision from model output, that decision should have been a tool call. Parsing free-form text to recover structured intent is a sign the structure belongs in the schema."*

It does not fit when the model can answer from training alone, when the interaction is one-shot with no side effects, or when *"tool-calling latency would dominate a trivial response."* That last one is quantitative and §31.19 quantifies it: every tool call is at least one extra round trip, and a round trip on a 20,000-token prefix is not free even when it hits the cache.

Note the overlap with Chapter 23. Structured outputs and a tool schema both produce a guaranteed shape; the difference is that a tool is a **request to do something** and a structured output is **the answer itself**. If nothing executes, you want `output_config.format`. If something executes, you want a tool. Chapter 23's rule still applies to both: *a grammar constrains the shape of an answer and says nothing about its truth.*

31.8 The Economics of a Tool-Using Turn

Three costs move together, and only the first is usually measured.

Round trips. `k` tools run in one wave when called in parallel and in `k` waves when not. For a 24-tool task, that is 4 requests against 25.

Tokens. Chapter 16's statelessness is unchanged and now driven by tool traffic: every request resends the entire history, so each extra round trip pays for everything that came before it. $31.19 measures 656,000 input tokens serially against 104,960 at a fan-out of eight — **6.25×** — on identical work.

Cache hits. Chapter 30's window is 20 blocks; a fan-out of `k` adds `2k`. The largest fan-out that keeps a single end-of-prompt breakpoint working is therefore **nine**, and the tenth tool takes the same task from 48,004 billed units to 100,140.

That third cost has a shape worth internalising, because it is not monotonic in either direction. Widening the fan-out reduces round trips and reduces tokens — right up to the point where it destroys the cache, after which it more than doubles them. The optimum is not "as parallel as possible" and not "one at a time". It is **as parallel as the cache window allows**, with a second breakpoint if you need more.

> TIP
>
> **The fix, if you need a wider fan-out**
>
> Chapter 30's remedy applies directly: add a **second cache breakpoint** nearer the growing end of the prompt, placed before the conversation grows, so a write accumulates there. Four breakpoints are available and *"cache breakpoints themselves don't add any cost."*

31.9 Architecture: Where the Loop Lives

The loop is application code, and it is the first place in this book where your system has a control flow of its own rather than a request and a response. That has three consequences.

It is **stateful in your process and stateless on the wire**. The message list is the state; the API holds nothing. Chapter 16's rule that statelessness is quadratic now compounds with tool traffic, and Chapter 54's compaction exists because of it.

It is **the boundary where authority is exercised**. Chapter 25 established that only the action layer prevents harm, and that it authorises the caller and never the model's request. The loop is that layer. The `tool_use` block is a proposal from a component that has read untrusted web pages; the decision to run it is yours, and it should be taken against the identity of the human or service on whose behalf the loop is running. Chapter 34 builds this properly.

It is **where retries have to be reasoned about twice**. Chapter 29's policy governs the HTTP call. It says nothing about whether re-running a tool is safe, and a retried request that re-sends a `tool_result` for a call your code already executed does not re-execute it — but a retried *turn* might. Chapter 34 takes up idempotency at the tool level; the loop's job is to make the distinction visible.

31.10 Figures



```
  ┌──────────┐                          ┌─────────┐
  messages[] ─────request──────────────► model
                                         │
                                         │ emits typed blocks
                                         ▼
  ┌──────────────────────────────────────────────────────┐
  │ assistant content                                    │
  │   [text] [thinking] [tool_use] [server_tool_use]     │
  └──────────────────────────────────────────────────────┘
       │                        │              │
       │                        │              └── Anthropic runs it;
       │                        │                  result arrives as a
       │                        │                  *_tool_result block
       │                        │                  (you build nothing)
       │                        ▼
       │              YOU run the code
       │                        │
       │                        ▼
       │              [tool_result tool_use_id=...]
       ▼                        │
  next request ◄────────────────┘    results FIRST, text after,
                                     one per id, same tools array
  stop_reason decides the loop:
    tool_use   -> run tools, continue      pause_turn -> resend
    end_turn / stop_sequence -> done       everything else -> HALT
```

```
one round trip at fan-out k appends 2k blocks:
   assistant: [tool_use][tool_use] ... k of them
   user:      [tool_result][tool_result] ... k of them
Chapter 30: a hit needs the previous write within 20 POSITIONS,
and the breakpoint counts as the first, so at most 19 new blocks.
   2k <= 19   ->   k <= 9
 k    blocks/trip   trips   cache hits   billed units
 8         16         4        3 / 4         47,848
 9         18         4        3 / 4         48,004     <- safe
10         20         4        1 / 4        100,140     <- +108.6%
12         24         3        0 / 3         98,400
Round trips do not change between k=9 and k=10. Latency does not
change. Only the cache changes, and it changes by 2.086x.
```

31.11 Examples

Example 1 — the loop, keyed on a table rather than one value.

```
while True:
    resp = client.messages.create(model=M, tools=TOOLS,
                                  max_tokens=4096, messages=msgs)
    action = LOOP_ACTION[resp.stop_reason]      # KeyError is correct
    if action == "DONE":
        return resp
    if action == "HALT":
        raise LoopHalted(resp.stop_reason)      # refusal, truncation
    if action == "RESEND":                      # pause_turn
        msgs.append({"role": "assistant", "content": resp.content})
        continue
    msgs.append({"role": "assistant", "content": resp.content})
    msgs.append({"role": "user", "content": run_all(resp.content)})
```

Example 2 — one user message, results first.

```
results = [{"type": "tool_result", "tool_use_id": b.id,
            "content": execute(b)}
           for b in resp.content if b.type == "tool_use"]
msgs.append({"role": "user", "content": results})    # not one each
```

Example 3 — a call you deliberately did not run.

```
{"type": "tool_result", "tool_use_id": "toolu_02", "is_error": True,
 "content": "Not executed: the preceding write_file call failed."}
```

Example 4 — disabling parallelism, in the right place.

```
tool_choice={"type": "auto", "disable_parallel_tool_use": True}
# auto -> at most one call; any/tool -> exactly one.
# NOT a top-level parameter. Changing it invalidates the message cache.
```

Example 5 — detecting the unresolved server tool (Chapter 28).

```
resolved = {b["tool_use_id"] for b in resp["content"]
            if b["type"].endswith("_tool_result")}
pending = [b["id"] for b in resp["content"]
           if b["type"] == "server_tool_use" and b["id"] not in resolved]
if pending:                          # send ONLY tool_result blocks back
    assert all(b["type"] == "tool_result" for b in next_user_content)
```

Example 6 — keeping the tools prefix cacheable (Chapter 30).

```
tools = [*BASE_TOOLS[:-1],
         {**BASE_TOOLS[-1],
          "cache_control": {"type": "ephemeral", "ttl": "1h"}}]
# Same array, same order, sorted keys, every turn. Any edit
# invalidates tools, system AND messages.
```

31.18 Code Walkthrough: A Loop That Refuses, and a Linter for What Is Legal

`agentloop.py` is two functions and a table. The table is the point.

```
LOOP_ACTION = {
    "tool_use": "RUN_TOOLS",
    "pause_turn": "RESEND",
    "end_turn": "DONE",
    "stop_sequence": "DONE",
    "max_tokens": "HALT",
    "refusal": "HALT",
    "model_context_window_exceeded": "HALT",
}
```

Seven keys and no default. `LOOP_ACTION[response["stop_reason"]]` raises `KeyError` on anything unlisted, which is the behaviour Chapter 26's versioning contract requires: Anthropic reserves the right

to *"add new variants to enum-like output values"*, and a loop that treats an unknown stop reason as "not `tool_use`, so we must be done" will one day exit on something that meant continue. Three of the seven are `HALT` rather than `DONE`, because a truncated turn, a refusal and an over-long context are all endings that need a decision from a layer above the loop rather than a `return`.

`validate_continuation` reproduces the documented 400s offline. The subtle branch is the one about trailing text, because trailing text is conditionally legal:

```
    first_non_result = next(
        (i for i, k in enumerate(kinds) if k != "tool_result"), None)
    if first_non_result is not None:
        if any(k == "tool_result" for k in kinds[first_non_result:]):
            raise Rejected(
                "tool_use ids were found without tool_result blocks "
                "immediately after: a %r block precedes a tool_result"
                % kinds[first_non_result])
        if unresolved:
            raise Rejected(
                "`web_search` tool use with id %r was found without a "
                "corresponding result block: text after the results "
                "ends the assistant turn" % unresolved[0])
```

The first branch is the unconditional rule — a non-result block before a result is always wrong. The second fires only when `reader.read()` reported an unresolved `server_tool_use`, which is the case where text **after** all the results is also wrong. A formatter that does not read the response cannot get this right, which is why the validator takes the response and not just the message it is about to send.

`batching_health` is the odd one out: it raises nothing, because there is nothing to raise.

```
        if n_calls > 1 and n_msgs > 1:
            problems.append(
                "turn %d: %d parallel calls answered across %d user "
                "messages; all results belong in ONE" % (i, n_calls,
                                                         n_msgs))
```

Every request in that history is valid and every response is a 200. The cost is that the history becomes a worked example of one-call-per-turn, and Chapter 5's induction heads describe exactly what a model does with a worked example in its context. This is the first linter in the book whose entire remit is **legal output that teaches the model the wrong habit**, and it is a category worth naming, because Chapter 33 will find more of them.

31.19 Mini Project: The Loop, the Gate, the Linter and the Fan-out Cliff

Folder structure.

```
ch31/
  agentloop.py    the loop table, the continuation gate, the linter
  fanout.py       round trips, tokens and cache hits vs fan-out
  cases.py        11 continuation cases and 2 histories, DOC or BUILT
  report.py       every published figure
  test_ch31.py    18 tests
```

```
  reader.py       copied from ch28/
  fixtures.py     copied from ch28/
  cacheplan.py    copied from ch30/
  costmodel.py    copied from ch11/
```

Result 1 — eleven continuation cases, six rejections, three UNKNOWNs

```
continuation cases  accept=5  reject=6  wrong=0
facts recorded UNKNOWN: 3
  UNKNOWN  the server-side loop's iteration cap inside a Message Batch
  UNKNOWN  whether tool_result content blocks count toward the
           20-block cache lookback individually or as one block
  UNKNOWN  the maximum number of tool_use blocks in one assistant turn
```

The six rejections are: text before the results, a missing result, an unknown `tool_use_id`, a duplicate result, a tool dropped from the `tools` array, and trailing text in a turn with an unresolved `server_tool_use`. The five acceptances include the last of those **without** the trailing text, which is the pair that proves the rule is conditional rather than absolute.

The second UNKNOWN is the one that should make a careful reader uncomfortable, because §31.19's headline result depends on it: the caching guide counts *blocks*, and a `tool_result` whose `content` is a list of three nested blocks might plausibly count as one position or three. This book's figures assume **one position per** `tool_result`, which is the reading that makes the cliff appear at a fan-out of nine; if the other reading is correct the cliff arrives sooner, never later. The finding is directionally safe and the exact threshold is stated with its assumption attached.

Result 2 — the history that is legal and still wrong

```
well-formed history  problems=[]
split-result history problems=1
   turn 1: 2 parallel calls answered across 2 user messages; all
           results belong in ONE
   good  history: API accepts turn 1 = True
   split history: API accepts turn 1 = False
```

Read the last two lines carefully. The **first** user message of the split history is rejected by the gate, because it answers two calls with one result — that failure is loud. The version that is genuinely dangerous is the one where each assistant turn made a single call and each user message answered it, so every request is well-formed and the model has simply been shown, turn after turn, that this is a one-call-at-a-time conversation. The linter is looking for the shape, not for an error.

Result 3 — the loop over all seven stop reasons

```
  end_turn                        -> DONE
  max_tokens                      -> HALT
  model_context_window_exceeded   -> HALT
  pause_turn                      -> RESEND
  refusal                         -> FALLBACK / HALT
  stop_sequence                   -> DONE
  tool_use                        -> RUN_TOOLS
  refusal                       halted: stop_reason 'refusal': FALLBACK
  max_tokens                    halted: stop_reason 'max_tokens':
                                TRUNCATED
```

```
  model_context_window_exceeded  halted: TRUNCATED
  an unlisted stop_reason raises rather than looping
```

Three of seven halt, one resends, two finish, and exactly **one** runs your tools. A loop keyed on `stop_reason == "tool_use"` alone gets four of the seven wrong in the sense that matters: it does something, quietly, that the response did not license.

Result 4 — the tenth parallel tool

Twenty-four tools to run, a 20,000-token cached prefix, one end-of-prompt breakpoint, 120 tokens per call and 400 per result:

```
largest fan-out that preserves one end-of-prompt breakpoint: 9
   k  blocks/trip  trips  hits  billed units   vs k=9
   1            2     25    24      102952.0   +114.5%
   2            4     13    12       71464.0    +48.9%
   4            8      7     6       55720.0    +16.1%
   8           16      4     3       47848.0     -0.3%
   9           18      4     3       48004.0     +0.0%
  10           20      4     1      100140.0   +108.6%
  12           24      3     0       98400.0   +105.0%
  20           40      3     1       68640.0    +43.0%
9 -> 10 parallel tools: round trips 4 -> 4 (unchanged), billed input
48004 -> 100140 (+108.6%)
```

The curve falls, bottoms out at a fan-out of eight or nine, and then **doubles in one step**. Round trips are identical at nine and ten. Latency is identical. The only thing that changed is that 2 × 10 = 20 blocks per round trip pushes the previous cache write exactly one position outside Chapter 30's lookback window, and from that turn onwards the conversation re-prefills its entire history on every request.

The tail is worth reading too. At `k = 20` the cost partially recovers, to +43.0 per cent, because 24 tools at a fan-out of 20 leaves a final wave of only four — eight blocks — which lands back inside the window. The function is not monotonic, is not smooth, and cannot be tuned by gradient descent on a dashboard. It has to be reasoned about.

Result 5 — what disabling parallelism costs

```
  fan-out  trips  latency(s)  uncached input tokens
        1     25        81.7                656000
        2     13        42.1                341120
        4      7        22.3                183680
        8      4        12.4                104960
       24      2         5.8                 52480
serial vs 8-wide: 6.25x the input tokens and 6.59x the wall clock
```

The same twenty-four tool calls, run one per turn, cost **6.25 times** the input tokens and **6.59 times** the wall clock. Nothing is wrong with any individual request. This is the price of the degradation in Result 2, and it is the reason a linter for a legal message shape earns its place in a build pipeline.

What the project establishes

Four numbers. **Nine** is the largest fan-out that preserves a single end-of-prompt cache breakpoint. **2.086×** is what the tenth tool costs. **6.25×** is what losing parallelism costs in tokens. And **one** is the

number of stop reasons that continue your loop, out of seven.

31.20 Hands-on Labs

Lab 1 — Build the loop table (success: your loop raises on `"tool_use_v2"`). Implement the seven-way table with no default and assert that an unlisted value raises rather than exiting.

Lab 2 — Reproduce the four continuation 400s (success: four distinct rejections, offline). Text before results; a missing result; a dropped tool; trailing text with an unresolved server tool.

Lab 3 — Find your own fan-out cliff (success: two adjacent `k` with a cost ratio above 2). Use your real per-call and per-result token counts and your model's minimum cacheable length.

Lab 4 — Lint a production transcript (success: a count of turns where parallel calls were answered across multiple user messages). Run `batching_health` over a week of stored histories.

Lab 5 — Measure your average fan-out (success: a number, and a judgement about whether it is too low). The documented check is `total_tool_calls / tool_call_messages`, which *"should be > 1.0 if parallel calls are working."*

31.21 Lab Solutions

Lab 1. The whole solution is refusing to write a default. `LOOP_ACTION[stop]` with no `.get()` is the correct implementation; wrapping it to raise a domain exception is a nicety.

Lab 2. Order matters in the gate: the tools-array check runs first, because a dropped tool invalidates the request regardless of how well the message is formed. Then block ordering, then id matching. Reproducing them in that order gives error messages that name the first real problem.

Lab 3. The condition is `2k ≤ lookback − 1`. With the documented 20-block window that is `k ≤ 9` for a single breakpoint; with a second breakpoint the constraint is per-window rather than per-conversation and the cliff moves. If your `tool_result` blocks nest multiple content blocks, count them individually — that is the conservative reading of the UNKNOWN in Result 1.

Lab 4. Expect false positives on turns where you deliberately answered a single call in a single message; the linter only flags `n_calls > 1` answered across `n_msgs > 1`, which is the shape the documentation names.

Lab 5. A ratio near 1.0 with tools that are plainly independent points at Result 2's history shape before it points at prompting. Fix the history first; it is free.

31.22 Best Practices

BEST PRACTICE

1. **Key the loop on a table of all seven stop reasons**, with no default and no fall-through.
2. **Treat `pause_turn` as the server's loop**, never as one of your tools waiting.
3. **Return every result in one user message**, results first, one per `tool_use_id`.
4. **Return `is_error: true` for calls you chose not to run** rather than omitting them.
5. **Keep the `tools` array byte-identical between turns** — same members, same order, sorted schema keys.
6. **Check for unresolved `server_tool_use` blocks by id**, because no stop reason reports them.
7. **Never execute a tool call from a `max_tokens` response**; it may be incomplete.
8. **Cap fan-out at nine** with one breakpoint, or add a second breakpoint before you widen it.
9. **Run independent read-only tools concurrently and stateful ones sequentially**, and say which is which in the tool's own documentation.
10. **Keep untrusted content inside `tool_result` blocks**, never in `system` or a plain user `text` block.
11. **Prefer an Anthropic-schema tool** (`bash`, `text_editor`, `memory`, `computer`) over a custom equivalent; the schema is trained-in.
12. **Put `disable_parallel_tool_use` inside `tool_choice`**, and do not vary it per request unless you are prepared to lose the message cache.
13. **Log average tool calls per tool-calling message** and alert when it approaches 1.0.
14. **Authorise the caller, not the call.** The loop is the action layer; Chapter 25's rule is unchanged.

31.23 Common Mistakes

COMMON MISTAKE

`while stop_reason == "tool_use"`

Symptom: occasional truncated answers, occasional silent empty responses, occasional unfinished server-tool work presented as final. **Cause:** four of the seven stop reasons are handled by falling out of the loop. **Fix:** a seven-way table with `HALT` for `max_tokens`, `refusal` and `model_context_window_exceeded`, and `RESEND` for `pause_turn`. **Detection:** feed the loop a scripted response for each stop reason and assert it does not return normally on any of the three halting ones.

COMMON MISTAKE

One user message per tool result

Symptom: average tool calls per turn drifts to 1.0; latency and cost rise; no errors anywhere. **Cause:** the history teaches the model that this conversation does one call at a time. **Fix:** one user message containing every result. **Detection:** a linter over stored transcripts; the API will never tell you.

COMMON MISTAKE

Text before the tool results

Symptom: `tool_use ids were found without tool_result blocks immediately after`. **Cause:** a friendly preamble prepended to the results array. **Fix:** results first; text, if any, after. **Detection:** assert `content[0]["type"] == "tool_result"` in the formatter.

COMMON MISTAKE

Trailing text in a turn with an unresolved server tool

Symptom: a 400 naming a `srvtoolu_...` id, on a message that works fine in other turns. **Cause:** the assistant turn called a client tool **and** a server tool; text after the results ends the turn before the server tool can run. **Fix:** when any `server_tool_use` id has no matching result, send only `tool_result` blocks. **Detection:** match ids in the response before building the reply.

COMMON MISTAKE

Editing the tools array mid-conversation

Symptom: `but no 'web_search' tool was provided`, or a total loss of cache hits. **Cause:** a dynamic tool list rebuilt per request, or a language that randomises JSON key order. **Fix:** a fixed list with sorted keys; use `defer_loading` and tool search to add tools without touching the prefix. **Detection:** hash the serialised `tools` array each turn and assert it is constant.

COMMON MISTAKE

Executing an incomplete tool call

Symptom: a tool runs with truncated or invalid arguments. **Cause:** `stop_reason: "max_tokens"` cut a `tool_use` block in half, or fine-grained tool streaming delivered invalid JSON — Chapter 17's `INVALID_JSON` contract. **Fix:** halt on `max_tokens`; validate `input` against the schema before executing. **Detection:** schema validation at the call site, not inside the tool.

COMMON MISTAKE

Widening fan-out to reduce round trips

Symptom: input costs double after a "performance" change that reduced request count. **Cause:** `2k` blocks per round trip crossed Chapter 30's 20-block window. **Fix:** cap at nine, or add a second cache breakpoint. **Detection:** `cache_read_input_tokens` collapsing to zero from a specific turn onwards.

31.24 Performance Considerations

Parallel execution buys latency only for the tools themselves; the model turn on either side is serial. §31.19's model makes that explicit: `waves × (model + tool) + model`, so a fan-out that halves the number of waves does not halve wall clock — at 0.8-second tools and 2.5-second model turns, going from one wave to two adds 3.3 seconds, not 0.8.

Throughput is bounded by round trips before it is bounded by anything else. Each round trip is a full request against the RPM bucket of Chapter 19, and each one resends the whole history against ITPM. A cached prefix removes the ITPM draw — Chapter 19's cache-aware rule — but not the RPM draw, so an agent that runs twenty-five short turns is twenty-five requests no matter how good its cache is.

Chapter 30's tool-use interactions still apply and are worth restating in a tool-use chapter: place `cache_control` on the **last tool** in the array to cache the whole tool-definitions prefix; for an `mcp_toolset`, place it on the toolset entry and the API applies it to the final expanded tool; and when a server tool runs with caching enabled, the API adds its own 5-minute breakpoint on the result, so you can see `ephemeral_5m_input_tokens` writes in a request whose every marker said `1h`.

31.25 Security Considerations

Every tool result is untrusted input. This is the single most important security fact in Part 6 and it is stated in the documentation rather than inferred: web pages, inbound email, user uploads and third-party APIs all arrive inside `tool_result` blocks, and *"an attacker who can influence it may embed instructions that try to redirect Claude."* Keep it in `tool_result` blocks precisely because that is the channel with the least authority — Chapter 21 established the precedence order, and Chapter 22 measured how readily ordinary payloads emit convincing markup.

The loop is the enforcement point, and it authorises the caller. A `tool_use` block is a proposal produced by a model that has been reading attacker-influenced text. Whether `delete_account(id=…)` runs must depend on the identity and permissions of the principal the loop is acting for, never on the fact that the model asked. Chapter 25's measurement stands: only the action layer prevents harm.

Least privilege applies to the tool list. Every tool you send is a capability you have granted for that request. A tool that is never called still costs tokens and still enlarges the attack surface; Chapter 30's `defer_loading` lets you keep the granted set small without losing the ability to grow it.

Do not let the model choose its own authorisation. A tool named `run_as_admin` with a boolean `admin` parameter delegates an access-control decision to a language model. Chapter 34 formalises the alternative.

`is_error` **is a channel to the model, not to your logs.** What you put in an error string is read by Claude and influences the next turn. Do not include stack traces, connection strings or internal hostnames.

31.26 Real Production Examples

A code-reading agent. Nine files per turn, not twelve. One breakpoint at the end of the prompt, a second before the fan-out was widened. `bash` and `text_editor` rather than custom equivalents, because the schemas are trained-in.

A customer-service assistant. Client tools for the CRM, `web_search` for public documentation. Every turn must check for an unresolved `server_tool_use` before replying, because a turn that reads the CRM *and* searches the web is exactly the mixed case that forbids trailing text.

A data pipeline. Tools with side effects, run strictly sequentially in the order they appear, stopping on first failure and returning `is_error: true` for the rest. The documented affordance is doing real work

here: the continuation is still legal, and Claude *"will reissue the call on the next turn."*

A research agent on the batch path. Chapter 30's batch worker runs the server loop with a higher, unpublished iteration cap; a `pause_turn` in a batch result is an unfinished turn to be resubmitted, not a failure to retry.

31.27 Certification Notes

Concept	Exam domain	Weight
The tool-use contract and the three execution buckets	D2 Tool Design & MCP Integration	18%
The client loop and all seven stop reasons	D5 Context Management & Reliability	15%
The server-side loop and `pause_turn`	D5 Context Management & Reliability	15%
Continuation formatting and the documented 400s	D2 Tool Design & MCP Integration	18%
Parallel tool use and `disable_parallel_tool_use`	D1 Agentic Architecture & Orchestration	27%
Fan-out against the cache window	D1 Agentic Architecture & Orchestration	27%
Tool results as untrusted input	D5 Context Management & Reliability	15%

CERTIFICATION NOTE

Exam Tips

- **The model never executes anything.** Any answer implying Claude runs your code is wrong.
- **Server tools need no** `tool_result` **from you**; client tools always do.
- `pause_turn` **is the server's loop**, never a client tool waiting.
- `tool_result` blocks come **first**; text may only follow, and not at all when a server tool is unresolved.
- **All results in one user message.** Separate messages are legal and degrade parallelism.
- `disable_parallel_tool_use` lives **inside** `tool_choice`: `auto` → at most one, `any`/`tool` → exactly one.
- The `tools` array must be **unchanged** between turns.
- Never execute a tool call from a `max_tokens` response.
- `is_error: true` **is the correct answer** for a call you chose not to run.
- The regex tell: if you are parsing free-form text for a decision, that decision should have been a tool call.

31.28 Twenty Practice Questions

1. Who executes a `bash` tool call?
2. What are the three fields on a `tool_use` block?
3. Which stop reason means the server's loop hit its cap?
4. A response has `stop_reason: "tool_use"` and a `server_tool_use` block with no result. What do you send back?
5. Why is text after the tool results sometimes legal and sometimes a 400?

6. Where does `disable_parallel_tool_use` go, and what does it do under `tool_choice: {"type": "any"}`?
7. What is wrong with returning each tool result in its own user message?
8. Your loop is `while stop_reason == "tool_use"`. Name three failures it handles incorrectly.
9. Why is an Anthropic-schema tool more reliable than an identical custom tool?
10. What happens if you drop a tool from `tools` between turns?
11. You run tools sequentially and the second fails. What goes in the continuation?
12. Why does widening fan-out from 9 to 10 double the input bill?
13. Which content-block types may appear inside a `tool_result`'s `content`?
14. What does `is_error: true` communicate, and to whom?
15. Give the documented sign that a task should have been a tool call.
16. Does prompt caching survive adding a tool through tool search?
17. What must never go in a `system` prompt that often arrives in a `tool_result`?
18. Your average tool calls per tool-calling message is 1.02. What do you check first?
19. Can a `max_tokens` response contain a tool call, and may you run it?
20. On the batch path, how many server-loop iterations run before `pause_turn`?

31.29 Answers

1. **You do.** `bash`, `text_editor`, `memory` and `computer` are Anthropic-schema **client** tools: Anthropic publishes the schema, your application executes. **Wrong distractors:** "Anthropic runs it" (that is `web_search`, `web_fetch`, `code_execution`, `tool_search`); "it runs in the code execution sandbox" (a different tool entirely).
2. `id`, `name`, `input`. The `id` is what a `tool_result` matches with `tool_use_id`.
3. `pause_turn`**.** Documented default of 10 iterations per request synchronously; higher and unpublished on the batch path.
4. A user message containing **only** `tool_result` blocks for the client tools, with the same `tools` array. The API runs the server tool on that request and the next response starts with its result.
5. After a purely client-tool turn, trailing text is fine. When a `server_tool_use` block is unresolved, text after the results ends the assistant turn early and the request fails with a 400 naming the server tool.
6. **Inside the** `tool_choice` **object**, not at the top level. Under `any` (or `tool`) it means **exactly one** call; under `auto` it means **at most one**.
7. It is legal, but it *"teaches Claude to avoid parallel calls."* Measured cost of the resulting collapse: 6.25× the input tokens and 6.59× the wall clock on a 24-tool task.
8. `max_tokens` (may contain an incomplete tool call, and the answer is truncated); `refusal` (needs a different model, and `content` may be empty); `pause_turn` (unfinished work treated as finished). `model_context_window_exceeded` is a fourth.
9. The schema is **trained-in**: *"Claude has been optimized on thousands of successful trajectories that use these exact tool signatures."*

10. A 400 of the form `but no 'web_search' tool was provided` — and, separately, the entire cache is invalidated, because `tools` is the first level of the prefix hierarchy.
11. A `tool_result` for **every** `tool_use` id. For the ones you did not run, `is_error: true` with a brief explanation such as *"Not executed: the preceding write_file call failed."*
12. `2k` blocks per round trip crosses the 20-block lookback window, so the previous cache write falls out of range and every subsequent request re-prefills the history. Round trips and latency are unchanged; only the cache changes, by **2.086×**.
13. `text`, `image`, `document` and `search_result`.
14. That the tool failed. It is read **by Claude**, which is why it must be informative about the failure and free of internal detail.
15. *"If you're writing a regex to extract a decision from model output, that decision should have been a tool call."*
16. **Yes.** A tool discovered through tool search arrives as a `tool_reference` block in the conversation history, leaving the `tools` prefix untouched.
17. Untrusted content — web pages, inbound email, user uploads, third-party API output. It belongs in `tool_result` blocks, the channel with the least authority.
18. The **history shape**: are parallel results being split across user messages? That is free to fix and is the documented first cause. Prompting is second.
19. **Yes it can, and no you may not.** The block may be incomplete.
20. **UNKNOWN.** The batch guide says the loop runs *"more iterations per turn"* than a synchronous request without publishing a number. The correct answer refuses to assume it is still ten.

31.30 Ten Practical Exercises

1. Implement the seven-way loop table and unit-test each branch.
2. Reproduce all six continuation rejections offline.
3. Add a formatter assertion that `content[0]["type"] == "tool_result"`.
4. Hash your serialised `tools` array every turn and alert on any change.
5. Measure average tool calls per tool-calling message in production.
6. Run `batching_health` over a month of transcripts and count the offending turns.
7. Compute your own maximum safe fan-out from your block counts.
8. Replace one custom tool with its Anthropic-schema equivalent and compare call reliability.
9. Add unresolved-`server_tool_use` detection by id matching to your reader.
10. Audit every tool for whether it authorises the caller or trusts the request.

31.31 Five Challenge Labs

1. **The adaptive fan-out.** Cap parallelism dynamically at the largest value that keeps the current turn inside the cache window, given the live block count, and show it beating a fixed cap on a mixed workload.

2. **The transcript replayer.** Replay a stored history through the gate and the linter, and produce a report of every legal-but-harmful shape it contains.
3. **The sequential executor with a legal continuation.** Run a dependent batch in order, stop on first failure, and emit a continuation that the gate accepts.
4. **The injection harness.** Build tool results containing plausible embedded instructions and measure whether your loop's authorisation layer is affected at all — it should not be.
5. **The round-trip budget.** Given a latency SLA, compute the maximum number of loop iterations you can afford and design a stopping rule; compare against Chapter 35's orchestration patterns when you reach them.

31.32 Ten Interview Questions

1. Who runs the code, and why does that matter for security?
2. Walk me through the loop, including every stop reason.
3. What is the difference between the client loop and the server-side loop?
4. Why are tool results untrusted?
5. When would you disable parallel tool use?
6. How can a valid, accepted request make your system slower and more expensive?
7. What is the relationship between tool fan-out and prompt caching?
8. Why prefer `bash` to a custom shell tool?
9. How do you know a server tool has not run yet?
10. When is a tool the wrong answer?

31.33 Summary

Part 6 opens with the mechanism that turns a text model into a system that acts, and the first thing to understand about it is how little the model is allowed to do. Claude emits a typed request — an `id`, a `name` and an `input` conforming to a schema you wrote — and then stops. Every consequence in the world happens because your code, or Anthropic's servers, chose to act on that request. The contract is small enough to describe in a paragraph, which is why the interesting failures are all in how it meets everything else in the API.

The loop is where those failures start. The documentation's canonical shape is a `while` keyed on `stop_reason`, and the version most people write keys on one value out of seven. Three of the remaining six are endings that need a decision from a layer above the loop rather than a quiet exit: a truncated turn that may contain half a tool call, a refusal whose `content` may be empty, and an over-long context. A fourth, `pause_turn`, is not your loop at all — it is Anthropic's server-side loop hitting an iteration cap, and treating it as completion presents unfinished work as an answer. The loop this chapter builds is a table with seven keys and no default, because Chapter 26's versioning contract permits new values and a fall-through is a bug with a delivery date.

The continuation rules are the other half. Results must immediately follow their calls, come first in the content array, appear one per id, and travel in a single user message with an unchanged `tools` array —

and trailing text is legal after a purely client-tool turn and a 400 after a turn with an unresolved server tool, which means a formatter that has not read the response cannot be correct. Six of those rules produce documented 400s and all six are reproduced offline in $31.19. The seventh produces nothing at all: returning each result in its own user message is entirely legal and teaches the model to stop batching, at a measured cost of 6.25 times the input tokens and 6.59 times the wall clock on identical work. It is the first failure in this book whose only possible detector is a linter you wrote yourself, and it will not be the last.

The chapter's sharpest number comes from a collision between two features documented on different pages. A parallel fan-out of `k` appends `2k` blocks per round trip, and Chapter 30's cache lookback window is twenty blocks, so nine is the largest fan-out that preserves a single end-of-prompt breakpoint. Widening from nine to ten leaves round trips and latency untouched and doubles the input bill — 48,004 base-input units to 100,140, an increase of 108.6 per cent — because the previous cache write lands exactly one position out of range and stays there for the rest of the conversation. The cost curve falls, bottoms out, doubles in one step and then partially recovers at very wide fan-outs when the final wave happens to be small. It is not smooth, not monotonic and not something a dashboard will tune for you. Chapters 32 to 35 take up the schema, the words, the error contract and the orchestration; all four inherit this chapter's rule that the model proposes and your system disposes.

KEY TAKEAWAYS

1. **The model never executes anything.** It emits a typed request; your code or Anthropic's servers act on it.
2. **Three buckets:** user-defined client, Anthropic-schema client (you still execute), and server-executed (you build no `tool_result`).
3. **One stop reason out of seven continues your loop.** Three halt, one resends, two finish, and an unlisted value must raise.
4. `pause_turn` **is the server's loop**, never a client tool waiting on you.
5. **Results first, one per id, one user message, unchanged** `tools` — and trailing text only when no server tool is unresolved.
6. `is_error: true` **is the right answer for a call you chose not to run**, and keeps the continuation legal.
7. **Splitting results across user messages is legal and teaches the model to stop batching**: 6.25× the tokens, 6.59× the wall clock.
8. `disable_parallel_tool_use` **lives inside** `tool_choice`**:** `auto` → at most one, `any/tool` → exactly one.
9. **A fan-out of** `k` **adds** `2k` **blocks per round trip, so nine is the cap** with one cache breakpoint; the tenth tool costs 2.086×.
10. **Every tool result is untrusted input**, and the loop — not the model — is the layer that authorises the caller.

31.34 Glossary

Term	Definition
Agentic loop	The client-driven cycle of request, `tool_use`, execute, `tool_result`; one assistant turn throughout
Anthropic-schema tool	A client-executed tool whose schema Anthropic publishes and the model is trained on: `bash`, `text_editor`, `memory`, `computer`
Client tool	Any tool your application executes and returns a `tool_result` for
`disable_parallel_tool_use`	A field **inside** `tool_choice` limiting Claude to at most (or exactly) one call
Fan-out	The number of `tool_use` blocks in one assistant turn
`is_error`	An optional `tool_result` field marking a failed or unrun call; read by the model
Parallel tool use	Several tool calls in one turn; on by default for Claude 4 and later
Round trip	One request/response pair in the loop; the unit of latency and of RPM draw
Server tool	A tool Anthropic executes: `web_search`, `web_fetch`, `code_execution`, `tool_search`
`server_tool_use`	The block recording a server-tool call; unresolved when no matching result block exists
Server-side loop	Anthropic's internal iteration over server tools, bounded by an iteration cap and reported by `pause_turn`
`tool_choice`	The object controlling whether and how Claude calls tools: `auto`, `any`, `tool`, `none`
`tool_result`	A user content block carrying `tool_use_id`, optional `content` and optional `is_error`
`tool_use`	An assistant content block carrying `id`, `name` and `input`
Tool-use contract	You declare the operations; Claude decides when to call them; execution is never the model's
Unresolved server tool	A `server_tool_use` id with no matching result block, detectable only by id matching

31.35 Further Reading

Primary sources, verified 2 August 2026 at `platform.claude.com/docs/en`:

- `agents-and-tools/tool-use/how-tool-use-works` — the contract, the three buckets, both loops, when not to use tools.
- `agents-and-tools/tool-use/handle-tool-calls` — `tool_use` and `tool_result` shapes, the formatting rules, `is_error`, the untrusted-content warning.
- `agents-and-tools/tool-use/parallel-tool-use` — execution semantics, `disable_parallel_tool_use`, and the troubleshooting section naming the split-message failure.
- `build-with-claude/handling-stop-reasons` — the seven stop reasons and the unresolved-server-tool contract, verified in Chapter 28.
- `agents-and-tools/tool-use/tool-use-with-prompt-caching` — breakpoints on tools and `defer_loading`, verified in Chapter 30.

Where to go next in this book. Chapter 32 specifies the schema this chapter has only referred to, including the supported JSON Schema subset and `strict: true`. **Chapter 33** covers the words — names, descriptions and affordances — that decide whether the right tool is called at all. **Chapter 34** builds the error contract and the idempotency this chapter's loop assumes. **Chapter 35** takes the loop up to many tools across many turns. **Chapter 36** opens Part 7 with why MCP exists, which is the question of where tools come from when they are not yours. **Chapter 54** covers compaction, the answer to a history that has grown too large to resend. **Chapter 58** treats prompt injection through tool results properly.